| SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE | DEPARTMENTOFCOMPUTER SCIENCE ENGINEERING |
|---|---|

| | | |
|---|---|---|
| **Program Name:** B. Tech | **AssignmentType: Lab** | **AcademicYear:**2025-2026 |
| **Course Coordinator Name** | Dr. Rishabh Mittal | |
| **Instructor(s)Name** | Mr. S Naresh Kumar<br>Ms. B. Swathi<br>Dr. Sasanko Shekhar Gantayat<br>Mr. Md Sallauddin<br>Dr. Mathivanan<br>Mr. Y Srikanth<br>Ms. N Shilpa<br>Dr. Rishabh Mittal (Coordinator)<br>Dr. R. Prashant Kumar<br>Mr. Ankushavali MD<br>Mr. B Viswanath<br>Ms. Sujitha Reddy<br>Ms. A. Anitha<br>Ms. M.Madhuri<br>Ms. Katherashala Swetha<br>Ms. Velpulasumalatha<br>Mr. Bingi Raju | |

| | | | |
|---|---|---|---|
| **CourseCode** | 23CS002PC304 | **CourseTitle** | AI Assisted Coding |
| **Year/Sem** | III/II | **Regulation** | R23 |
| **DateandDay of Assignment** | Week7 - Tuesday | **Time(s)** | |
| **Duration** | 2 Hours | **Applicable to Batches** | 23CSBTB01 To 23CSBTB52 |

**AssignmentNumber:13.2**(Presentassignmentnumber)/**24**(Totalnumberofassignments)

| **Q.No.** | **Question** | ***Expected Time to complete*** |
|---|---|---|
| 1 | **Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions**<br>**Lab Objectives:**<br>• Identify code smells and inefficiencies in legacy Python scripts. | Week 7 - Tuesday |

- Use AI-assisted coding tools to **refactor** for readability, maintainability, and performance.
- Apply **modern Python best practices** while ensuring output correctness.

**Task Description -1 (Refactoring – Eliminating Repetitive Logic)**

- Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

**Instructions:**

- Ask AI to locate repeated computations within the program.
- Encapsulate repeated logic into reusable functions.
- Verify that program output remains unchanged after refactoring.
- Add proper docstrings to all defined functions.

**Sample Code:**

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

**Expected Output:**

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

## Step 1: Identify the Problem

The code repeats the **same area and perimeter calculation** multiple times.

## Step 2: Create a Function

Move the repeated logic into a reusable function.

## Step 3: Call the Function for Different Values

## Refactored Code

```
def calculate_rectangle(length, width):
```

```
    """
    Function to calculate area and perimeter of a rectangle
    """
    area = length * width
    perimeter = 2 * (length + width)

    print("Area:", area)
    print("Perimeter:", perimeter)


# First rectangle
calculate_rectangle(8, 4)

# Second rectangle
calculate_rectangle(12, 6)
```

**CODE OUTPUT**

Area: 32
Perimeter: 24
Area: 72
Perimeter: 36

## Explanation

- Created function **calculate_rectangle()**
- Eliminates duplicate code
- Improves readability and reuse

**Task Description -2 (Refactoring – Modularizing Inline Calculations)**

- Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

**Instructions:**

- Identify repeated calculation patterns in the code.
- Convert them into reusable functions.
- Ensure readability and proper documentation.

**Sample Code:**

amount = 1000
discount = amount * 0.10

final_amount = amount - discount
print("Final Amount:", final_amount)

amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)

**Expected Output:**

A modular Python program that uses a reusable function to compute final amounts for different inputs.

## Step 1: Identify Repeated Logic

Discount calculation is repeated.

## Step 2: Create Function

Create a reusable function for calculating final amount.

### Refactored Code

```
def calculate_final_amount(amount):
    discount = amount * 0.10
    final_amount = amount - discount
    print("Final Amount:", final_amount)


calculate_final_amount(1000)
calculate_final_amount(2000)
```

### CODE OUTPUT

Final Amount: 900.0
Final Amount: 1800.0

### Explanation

- Created function **calculate_final_amount()**
- Avoids repeating discount logic.

**Task Description -3 (Refactoring – Improving Loop and Conditional Performance)**

- Task: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.

**Instructions:**

- Use AI to recommend more efficient data structures or logic simplifications.
- Replace inefficient constructs while preserving program behavior.
- Compare execution efficiency before and after refactoring.

**Sample Code:**

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
print(name, "not found")
```

**Expected Output:**

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

## Step 1: Identify Inefficient Logic

The code uses **nested loops**, which is slow.

## Step 2: Use Python Set

Set lookup is **faster (O(1))** than list search.

## Refactored Code

```
students = {"Anil", "Bhavya", "Charan", "Divya"}
```

```python
check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:
    if name in students:
        print(name, "found")
    else:
        print(name, "not found")
```

**CODE OUTPUT**

Charan found
Esha not found
Bhavya found

**Explanation**

- Converted list → **set**
- Removed nested loop
- Faster lookup.

**Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)**

- Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

**Instructions:**

- Detect hardcoded numeric values used directly in expressions.
- Define descriptive constants at the beginning of the program.
- Update calculations to use these constants without changing results.

**Sample Code:**

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

**Expected Output:**

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations..

## Step 1: Identify Hardcoded Values

Numbers **2, 5, 100** are directly used.

## Step 2: Create Constants

### Refactored Code

```
RATE = 2
TIME = 5

def calculate_simple_interest(principal):
    interest = (principal * RATE * TIME) / 100
    print("Simple Interest:", interest)


calculate_simple_interest(1000)
calculate_simple_interest(2000)
```

### CODE OUTPUT

Simple Interest: 100.0

Simple Interest: 200.0

### Explanation

- Defined constants **RATE** and **TIME**
- Makes code easier to maintain.

**Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)**

- Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments..

**Instructions:**

- Replace non-descriptive variable names with meaningful identifiers.
- Add inline comments where the logic may be unclear.

- Ensure program output remains the same.

**Sample Code:**

x = 50
y = 30
z = x + y
print(z)

**Expected Output:**

A Python program with clear variable names and explanatory comments that improve readability without altering functionality.

## Step 1: Identify Poor Variable Names

Variables **x, y, z** are unclear.

## Step 2: Rename Variables

### Refactored Code

```
# First number
num1 = 50

# Second number
num2 = 30

# Calculate sum
total_sum = num1 + num2

# Display result
print(total_sum)
```

**CODE OUTPUT**

**80**

### Explanation

- Renamed variables to **num1, num2, total_sum**
- Added comments for clarity.

| | | |
|---|---|---|
| | **Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots** | |